

Module 1

November 9, 2024

```
[14]: import pandas as pd
import numpy as np
```

```
[3]: gunnels = pd.read_csv("https://raw.githubusercontent.com/salemprakash/EDA/main/
↳Data/Gunnels.csv")
print(gunnels)
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
0	1	0	786	190	10	0.1	0	3	0	
1	2	0	794	182	10	0.0	0	3	0	
2	3	0	798	178	20	0.0	0	1	0	
3	4	0	800	176	70	0.3	0	1	0	
4	5	0	804	172	40	0.3	0	1	0	
...	...	...	...	...	...	...	...	...	...	...
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	
1591	1592	0	796	148	20	1.0	0	1	1	

	Water	Cobble
0	0	0
1	0	0
2	0	0
3	1	0
4	1	0
...	...	...
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

[1592 rows x 11 columns]

```
[4]: gunnels.head()
```

```
[4]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	\
0	1	0	786	190	10	0.1	0	3	0	0	
1	2	0	794	182	10	0.0	0	3	0	0	
2	3	0	798	178	20	0.0	0	1	0	0	
3	4	0	800	176	70	0.3	0	1	0	1	
4	5	0	804	172	40	0.3	0	1	0	1	

```

Cobble
0      0
1      0
2      0
3      0
4      0

```

```
[5]: gunnels.head(10)
```

```
[5]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	\
0	1	0	786	190	10	0.1	0	3	0	0	
1	2	0	794	182	10	0.0	0	3	0	0	
2	3	0	798	178	20	0.0	0	1	0	0	
3	4	0	800	176	70	0.3	0	1	0	1	
4	5	0	804	172	40	0.3	0	1	0	1	
5	6	0	808	168	60	0.3	0	1	0	1	
6	7	0	814	162	40	0.3	0	1	0	1	
7	8	0	817	159	0	0.9	1	1	1	1	
8	9	0	821	155	50	0.5	0	1	0	1	
9	10	0	826	150	20	0.1	0	1	0	0	

```

Cobble
0      0
1      0
2      0
3      0
4      0
5      0
6      0
7      0
8      0
9      0

```

```
[8]: gunnels.tail()
```

```
[8]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	

1591	1592	0	796	148	20	1.0	0	1	1
------	------	---	-----	-----	----	-----	---	---	---

	Water	Cobble
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

```
[10]: gunnels.tail(10)
```

```
[10]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
1582	1583	1	763	115	10	1.0	3	8	0	
1583	1584	0	768	120	0	1.0	1	1	1	
1584	1585	0	770	122	0	1.0	2	1	0	
1585	1586	0	775	127	0	1.0	0	1	0	
1586	1587	0	778	130	0	1.0	0	1	0	
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	
1591	1592	0	796	148	20	1.0	0	1	1	

	Water	Cobble
1582	1	1
1583	1	0
1584	1	0
1585	1	0
1586	1	0
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

```
[11]: gunnels.shape
```

```
[11]: (1592, 11)
```

```
[13]: gunnels.dtypes
```

```
[13]: rownames      int64
      Gunnel        int64
      Time          int64
      Fromlow       int64
      Slope          int64
      Rw            float64
```

```
Amphiso      int64
Subst        int64
Pool         int64
Water        int64
Cobble       int64
dtype: object
```

```
[18]: mean_time = gunnels['Time'].mean()
      mean_time
```

```
[18]: np.float64(705.2437185929648)
```

```
[19]: mean_time = np.mean(gunnels['Slope'])
      mean_time
```

```
[19]: np.float64(9.503768844221106)
```

```
[ ]:
```

Module 2

November 9, 2024

```
[1]: import pandas as pd
import numpy as np
```

```
[3]: gunnels = pd.read_csv("https://raw.githubusercontent.com/salemprakash/EDA/main/
↳Data/Gunnels.csv")
print(gunnels)
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
0	1	0	786	190	10	0.1	0	3	0	
1	2	0	794	182	10	0.0	0	3	0	
2	3	0	798	178	20	0.0	0	1	0	
3	4	0	800	176	70	0.3	0	1	0	
4	5	0	804	172	40	0.3	0	1	0	
...	...	...	...	...	...	...	...	...	...	...
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	
1591	1592	0	796	148	20	1.0	0	1	1	

	Water	Cobble
0	0	0
1	0	0
2	0	0
3	1	0
4	1	0
...	...	...
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

[1592 rows x 11 columns]

```
[4]: gunnels.describe()
```

```
[4]:
```

	rownames	Gunnel	Time	Fromlow	Slope \
count	1592.000000	1592.000000	1592.000000	1592.000000	1592.000000
mean	796.500000	0.030151	705.243719	149.408920	9.503769
std	459.715129	0.171056	135.853638	82.662326	15.889862
min	1.000000	0.000000	340.000000	0.000000	0.000000
25%	398.750000	0.000000	623.000000	71.750000	0.000000
50%	796.500000	0.000000	712.500000	158.000000	0.000000
75%	1194.250000	0.000000	810.000000	224.250000	10.000000
max	1592.000000	1.000000	983.000000	304.000000	90.000000

	Rw	Amphiso	Subst	Pool	Water \
count	1592.000000	1592.000000	1592.000000	1592.000000	1592.000000
mean	0.527073	0.464196	3.105528	0.202261	0.508794
std	0.421278	0.899039	2.787720	0.401812	0.500080
min	0.000000	0.000000	1.000000	0.000000	0.000000
25%	0.000000	0.000000	1.000000	0.000000	0.000000
50%	0.600000	0.000000	2.000000	0.000000	1.000000
75%	1.000000	1.000000	4.000000	0.000000	1.000000
max	1.000000	4.000000	13.000000	1.000000	1.000000

	Cobble
count	1592.000000
mean	0.190955
std	0.393177
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

```
[5]: gunnels['Subst'].value_counts()
```

```
[5]: Subst
1      792
3      188
8      166
4      147
2      100
7       86
5       50
10      20
6       11
9       11
12      10
13       7
11       4
Name: count, dtype: int64
```

```
[6]: gunnels.isnull()
```

```
[6]:      rownames  Gunnel  Time  Fromlow  Slope    Rw  Amphiso  Subst  Pool  \
0      False  False  False   False  False  False   False   False  False
1      False  False  False   False  False  False   False   False  False
2      False  False  False   False  False  False   False   False  False
3      False  False  False   False  False  False   False   False  False
4      False  False  False   False  False  False   False   False  False
...
1587   False  False  False   False  False  False   False   False  False
1588   False  False  False   False  False  False   False   False  False
1589   False  False  False   False  False  False   False   False  False
1590   False  False  False   False  False  False   False   False  False
1591   False  False  False   False  False  False   False   False  False
```

```
      Water  Cobble
0      False  False
1      False  False
2      False  False
3      False  False
4      False  False
...
1587  False  False
1588  False  False
1589  False  False
1590  False  False
1591  False  False
```

```
[1592 rows x 11 columns]
```

```
[7]: gunnels.duplicated()
```

```
[7]: 0      False
1      False
2      False
3      False
4      False
...
1587   False
1588   False
1589   False
1590   False
1591   False
Length: 1592, dtype: bool
```

```
[9]: de_dup = gunnels.drop_duplicates()
de_dup
```

```
[9]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
0	1	0	786	190	10	0.1	0	3	0	
1	2	0	794	182	10	0.0	0	3	0	
2	3	0	798	178	20	0.0	0	1	0	
3	4	0	800	176	70	0.3	0	1	0	
4	5	0	804	172	40	0.3	0	1	0	
...	...	...	...	...	...	...	...	...	...	...
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	
1591	1592	0	796	148	20	1.0	0	1	1	

	Water	Cobble
0	0	0
1	0	0
2	0	0
3	1	0
4	1	0
...	...	...
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

[1592 rows x 11 columns]

```
[10]: gunnels.isnull().sum().sum()
```

```
[10]: np.int64(0)
```

```
[11]: gunnels.mean()
```

```
[11]: rownames    796.500000
      Gunnel      0.030151
      Time       705.243719
      Fromlow    149.408920
      Slope       9.503769
      Rw         0.527073
      Amphiso    0.464196
      Subst      3.105528
      Pool       0.202261
      Water      0.508794
      Cobble     0.190955
      dtype: float64
```

```
[12]: gunnels.cumsum()
```

```
[12]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
0	1	0	786	190	10	0.1	0	3	0	
1	3	0	1580	372	20	0.1	0	6	0	
2	6	0	2378	550	40	0.1	0	7	0	
3	10	0	3178	726	110	0.4	0	8	0	
4	15	0	3982	898	150	0.7	0	9	0	
...	...	...	...	...	...	...	...	...	...	...
1587	1261666	48	1119585	237288	15090	835.1	738	4934	320	
1588	1263255	48	1120369	237424	15100	836.1	738	4935	321	
1589	1264845	48	1121159	237566	15110	837.1	738	4936	321	
1590	1266436	48	1121952	237711	15110	838.1	739	4943	321	
1591	1268028	48	1122748	237859	15130	839.1	739	4944	322	

	Water	Cobble
0	0	0
1	0	0
2	0	0
3	1	0
4	2	0
...	...	...
1587	806	303
1588	807	303
1589	808	303
1590	809	304
1591	810	304

[1592 rows x 11 columns]

```
[14]: gunnels['Gunnel'].value_counts()
```

```
[14]: Gunnel
0    1544
1     48
Name: count, dtype: int64
```

```
[24]: word_gunnels = gunnels.copy()
word_gunnels['Gunnel'] = gunnels['Gunnel'].replace(to_replace=[0,1],
↪value=[False, True])
word_gunnels[1280:1290]
```

```
/tmp/ipykernel_60498/2823540284.py:2: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
word_gunnels['Gunnel'] = gunnels['Gunnel'].replace(to_replace=[0,1],
value=[False, True])
```

```
[24]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
1280	1281	False	855	247	0	0.1	0	1	0	
1281	1282	False	856	248	0	0.0	0	1	0	
1282	1283	False	857	249	0	0.0	0	1	0	
1283	1284	True	687	26	0	1.0	0	8	1	
1284	1285	False	691	30	10	0.9	0	8	1	
1285	1286	True	695	34	0	0.9	0	8	0	
1286	1287	False	697	36	0	0.1	0	8	1	
1287	1288	False	700	39	10	0.3	1	8	0	
1288	1289	False	702	41	0	0.9	0	8	1	
1289	1290	True	706	45	0	0.9	0	8	0	

	Water	Cobble
1280	1	0
1281	0	0
1282	0	0
1283	1	1
1284	1	1
1285	0	1
1286	1	1
1287	1	1
1288	1	1
1289	1	1

```
[39]: time_vals = gunnels['Time']
time_bins = pd.cut(time_vals, [600, 700, 800, 900, 1000])
time_bins
```

```
[39]: 0      (700, 800]
1      (700, 800]
2      (700, 800]
3      (700, 800]
4      (800, 900]
...
1587   (700, 800]
1588   (700, 800]
1589   (700, 800]
1590   (700, 800]
1591   (700, 800]
Name: Time, Length: 1592, dtype: category
Categories (4, interval[int64, right]): [(600, 700] < (700, 800] < (800, 900] < (900, 1000]]
```

```
[40]: time_bins.value_counts()
```

```
[40]: Time
(600, 700]      450
```

```
(700, 800]      392
(800, 900]      345
(900, 1000]      92
Name: count, dtype: int64
```

```
[41]: time_vals.quantile([0.25, 0.5, 0.75])
```

```
[41]: 0.25      623.0
      0.50      712.5
      0.75      810.0
      Name: Time, dtype: float64
```

```
[55]: IQR = np.percentile(time_vals, 75) - np.percentile(time_vals, 25)
      IQR
      val1 = np.percentile(time_vals, 25) - 1.5 * IQR
      val2 = np.percentile(time_vals, 75) + 1.5 * IQR

      time_vals[~time_vals.between(val1, val2)] # Outliers
```

```
[55]: 100      340
      101      341
      Name: Time, dtype: int64
```

```
[57]: non_outlier_time_vals = time_vals[time_vals.between(val1, val2)]
      non_outlier_time_vals #Without outliers
```

```
[57]: 0          786
      1          794
      2          798
      3          800
      4          804
      ...
      1587       782
      1588       784
      1589       790
      1590       793
      1591       796
      Name: Time, Length: 1590, dtype: int64
```

```
[ ]:
```

Module 3

November 9, 2024

```
[2]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
[3]: gunnels = pd.read_csv("https://raw.githubusercontent.com/salemprakash/EDA/main/
↳Data/Gunnels.csv")
print(gunnels)
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	\
0	1	0	786	190	10	0.1	0	3	0	
1	2	0	794	182	10	0.0	0	3	0	
2	3	0	798	178	20	0.0	0	1	0	
3	4	0	800	176	70	0.3	0	1	0	
4	5	0	804	172	40	0.3	0	1	0	
...	...	...	...	...	...	...	...	...	...	...
1587	1588	0	782	134	10	1.0	0	1	0	
1588	1589	0	784	136	10	1.0	0	1	1	
1589	1590	0	790	142	10	1.0	0	1	0	
1590	1591	0	793	145	0	1.0	1	7	0	
1591	1592	0	796	148	20	1.0	0	1	1	

	Water	Cobble
0	0	0
1	0	0
2	0	0
3	1	0
4	1	0
...	...	...
1587	0	0
1588	1	0
1589	1	0
1590	1	1
1591	1	0

[1592 rows x 11 columns]

```
[4]: gunnels.describe()
```

```
[4]:
```

	rownames	Gunnel	Time	Fromlow	Slope \
count	1592.000000	1592.000000	1592.000000	1592.000000	1592.000000
mean	796.500000	0.030151	705.243719	149.408920	9.503769
std	459.715129	0.171056	135.853638	82.662326	15.889862
min	1.000000	0.000000	340.000000	0.000000	0.000000
25%	398.750000	0.000000	623.000000	71.750000	0.000000
50%	796.500000	0.000000	712.500000	158.000000	0.000000
75%	1194.250000	0.000000	810.000000	224.250000	10.000000
max	1592.000000	1.000000	983.000000	304.000000	90.000000

	Rw	Amphiso	Subst	Pool	Water \
count	1592.000000	1592.000000	1592.000000	1592.000000	1592.000000
mean	0.527073	0.464196	3.105528	0.202261	0.508794
std	0.421278	0.899039	2.787720	0.401812	0.500080
min	0.000000	0.000000	1.000000	0.000000	0.000000
25%	0.000000	0.000000	1.000000	0.000000	0.000000
50%	0.600000	0.000000	2.000000	0.000000	1.000000
75%	1.000000	1.000000	4.000000	0.000000	1.000000
max	1.000000	4.000000	13.000000	1.000000	1.000000

	Cobble
count	1592.000000
mean	0.190955
std	0.393177
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

```
[31]: gunnels['Amphiso'].value_counts() # Close enough to a categorical data
```

```
[31]: Amphiso
0    1186
2     169
1     165
3      52
4      20
Name: count, dtype: int64
```

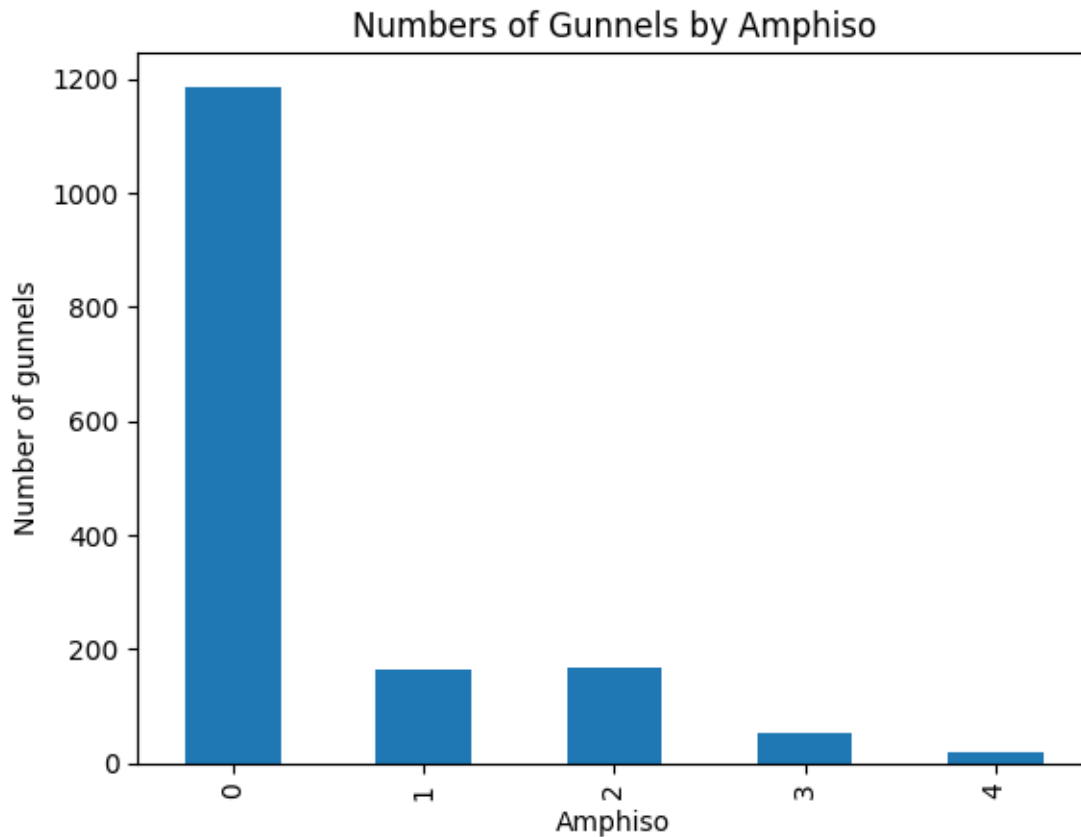
```
[32]: gunnels['Amphiso'].value_counts().sort_index()
```

```
[32]: Amphiso
0    1186
1     165
2     169
3      52
```

```
4      20
Name: count, dtype: int64
```

```
[33]: gunnels['Amphiso'].value_counts().sort_index().plot(kind='bar')
plt.title('Numbers of Gunnels by Amphiso')
plt.xlabel('Amphiso')
plt.ylabel('Number of gunnels')
```

```
[33]: Text(0, 0.5, 'Number of gunnels')
```



```
[36]: gunnels['Time'].value_counts()
```

```
[36]: Time
827    12
825    10
788    10
648    10
632    10
...
966     1
```

```

969      1
970      1
973      1
683      1
Name: count, Length: 542, dtype: int64

```

```
[38]: sns.FacetGrid(gunnels).map(sns.distplot,"Time").add_legend()
```

```

/files/Programming/College/EDA/EDA-DA/venv/lib/python3.12/site-
packages/seaborn/axisgrid.py:854: UserWarning:

```

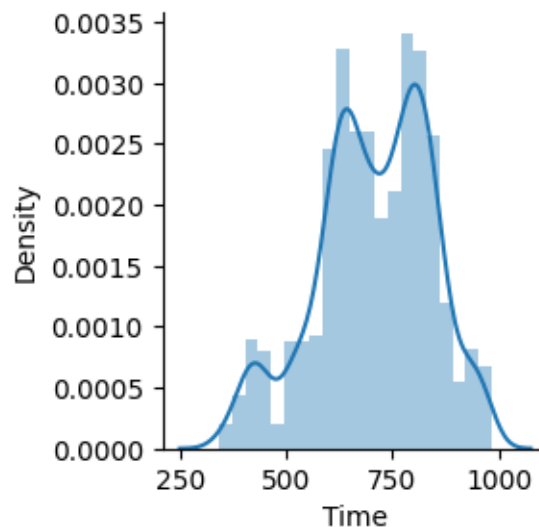
``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
func(*plot_args, **plot_kwargs)
```

```
[38]: <seaborn.axisgrid.FacetGrid at 0x7fcf8353b5f0>
```



```
[41]: sns.FacetGrid(gunnels).map(sns.distplot,"Fromlow").add_legend()
```

```

/files/Programming/College/EDA/EDA-DA/venv/lib/python3.12/site-
packages/seaborn/axisgrid.py:854: UserWarning:

```

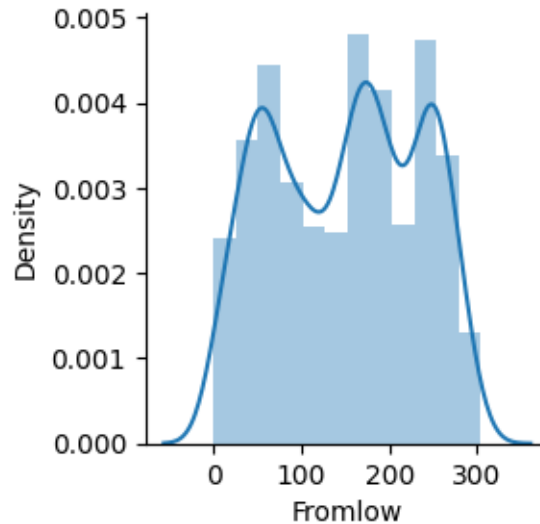
``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

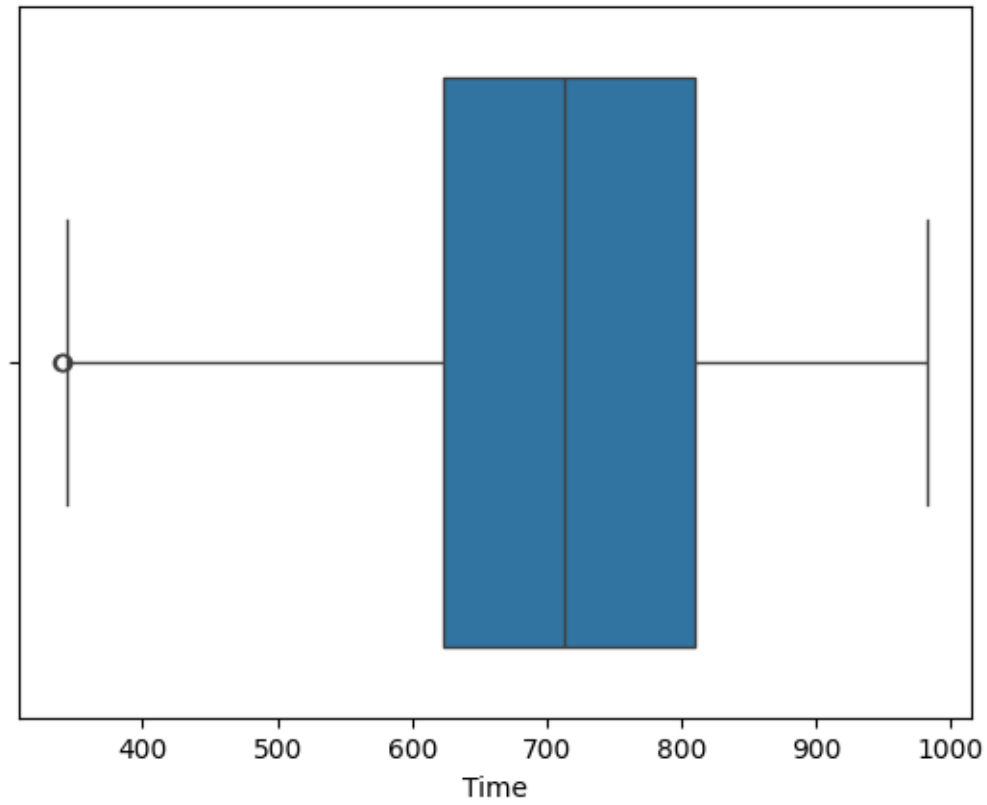
For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
func(*plot_args, **plot_kwargs)
```

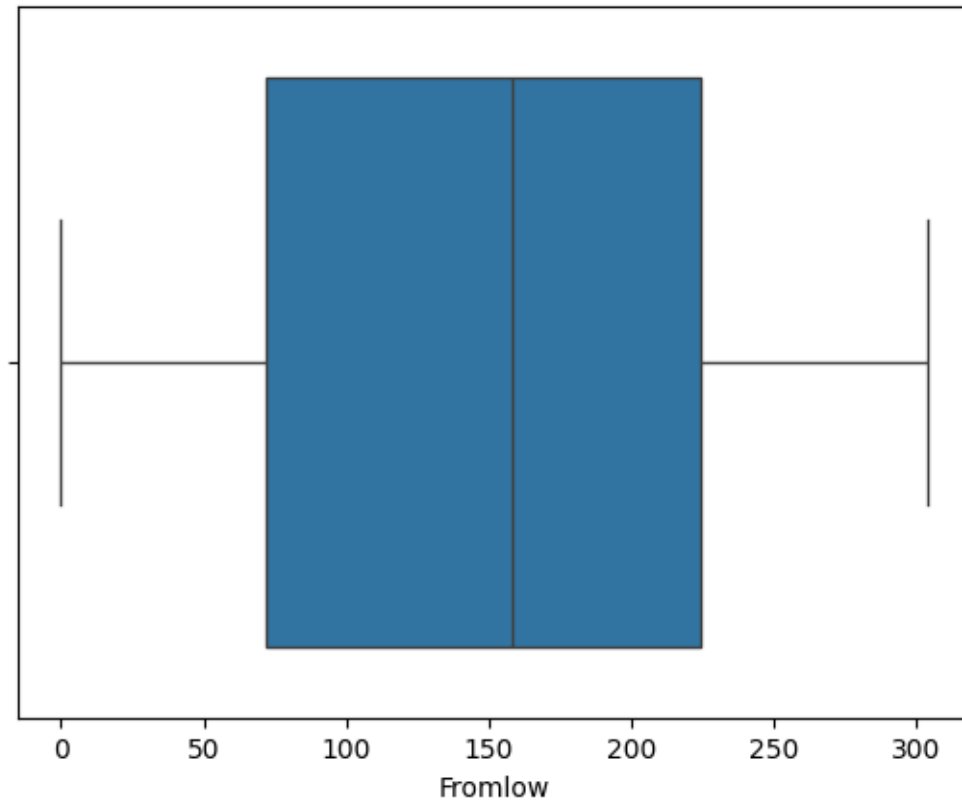
```
[41]: <seaborn.axisgrid.FacetGrid at 0x7fcf834419a0>
```



```
[43]: sns.boxplot(x="Time", data=gunnels)
plt.show() # Outliers exist
```

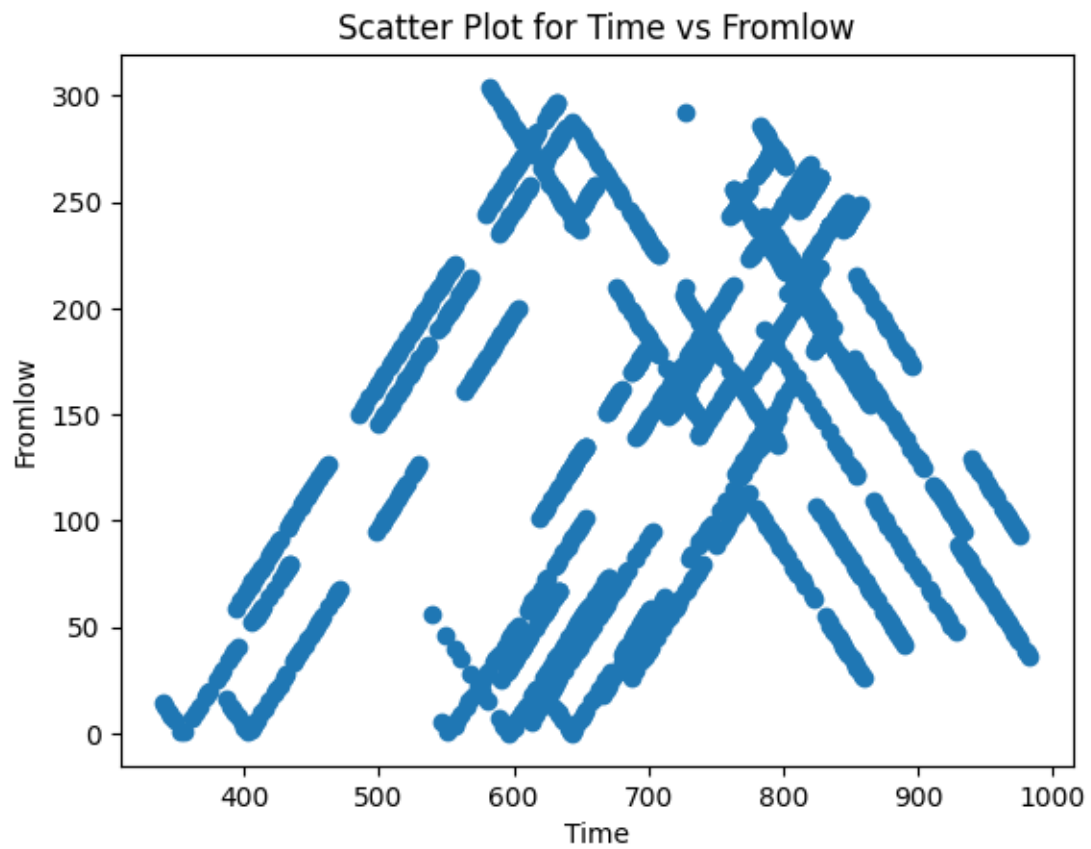


```
[44]: sns.boxplot(x="Fromlow", data=gunnels)
plt.show() # No outliers
```

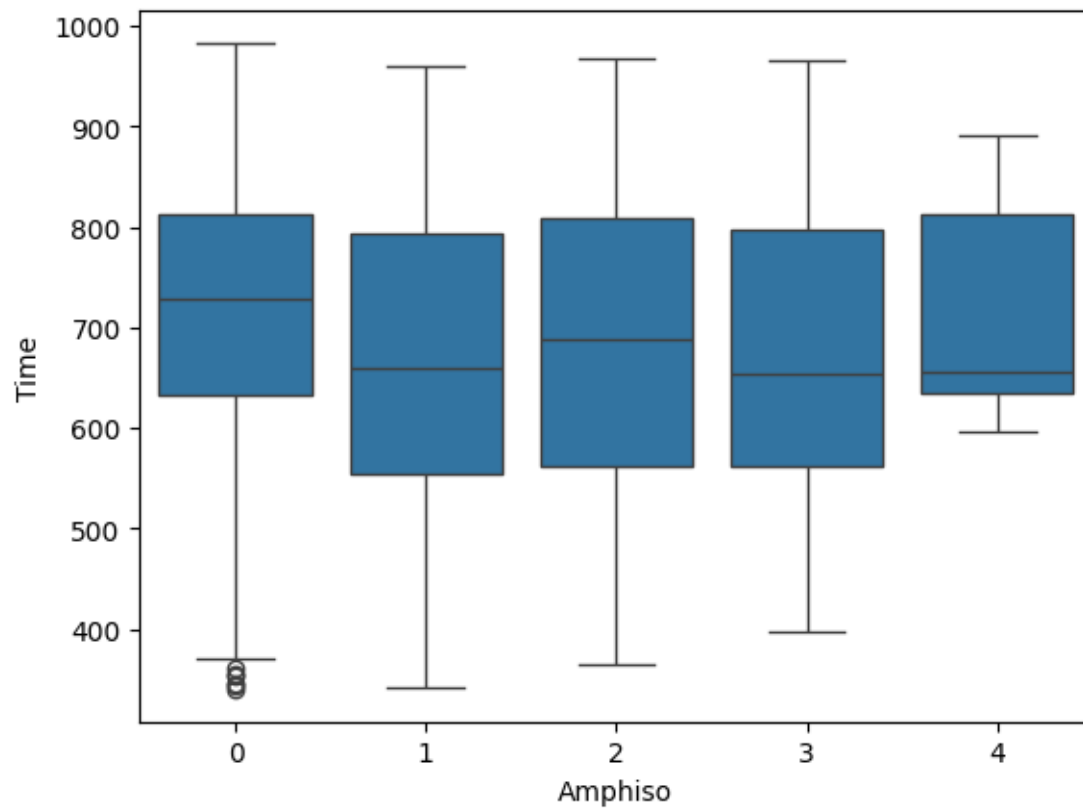


```
[50]: plt.scatter(gunnels["Time"], gunnels["Fromlow"])  
plt.title("Scatter Plot for Time vs Fromlow")  
plt.xlabel('Time')  
plt.ylabel('Fromlow')
```

```
[50]: Text(0, 0.5, 'Fromlow')
```

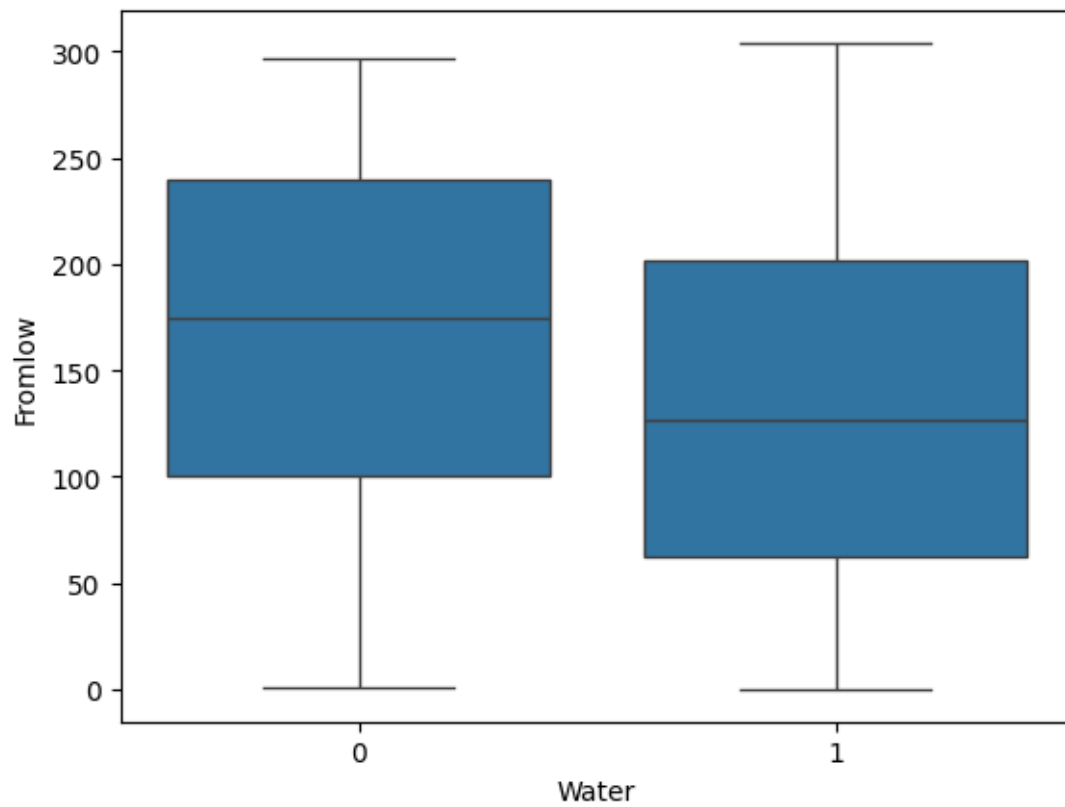


```
[54]: sns.boxplot(x="Amphiso",y="Time",data=gunnels)
plt.show() # Lots of outliers when Amphiso=0
```



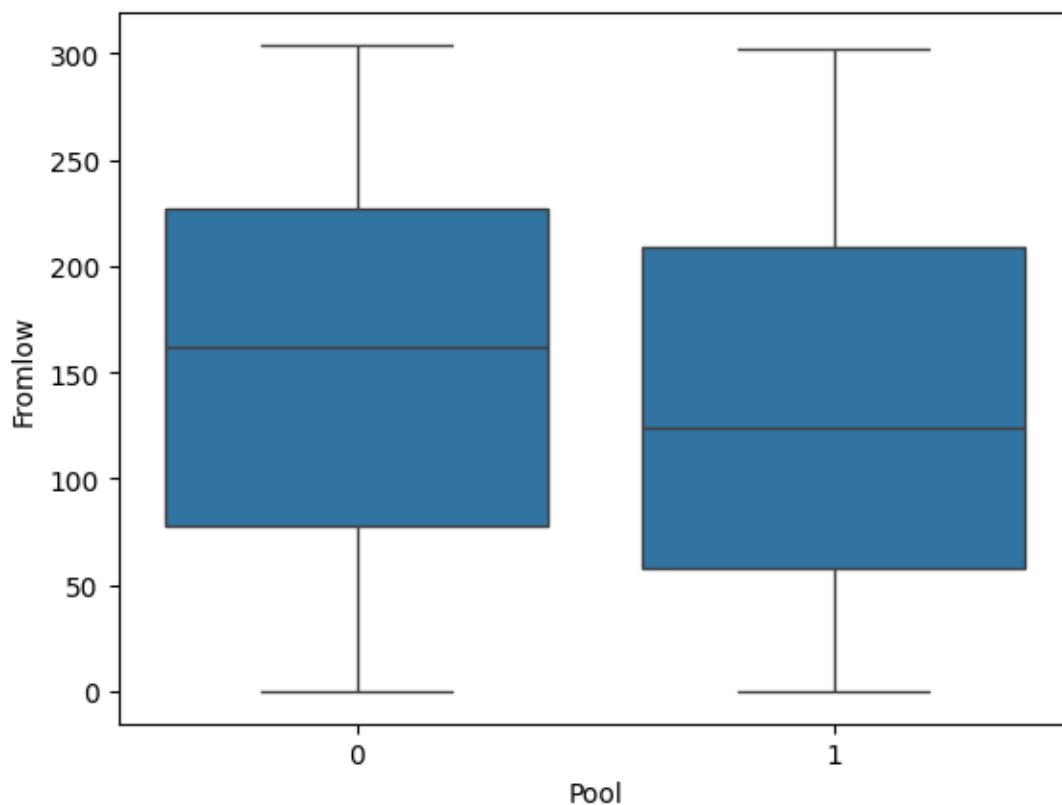
```
[62]: sns.boxplot(x="Water", y="Fromlow",data=gunnels)
```

```
[62]: <Axes: xlabel='Water', ylabel='Fromlow'>
```



```
[63]: sns.boxplot(x="Pool", y="Fromlow",data=gunnels) # Very similar to previous graph
```

```
[63]: <Axes: xlabel='Pool', ylabel='Fromlow'>
```



```
[65]: correlation = gunnels.corr(method='pearson', numeric_only=True)
      correlation
```

```
[65]:
```

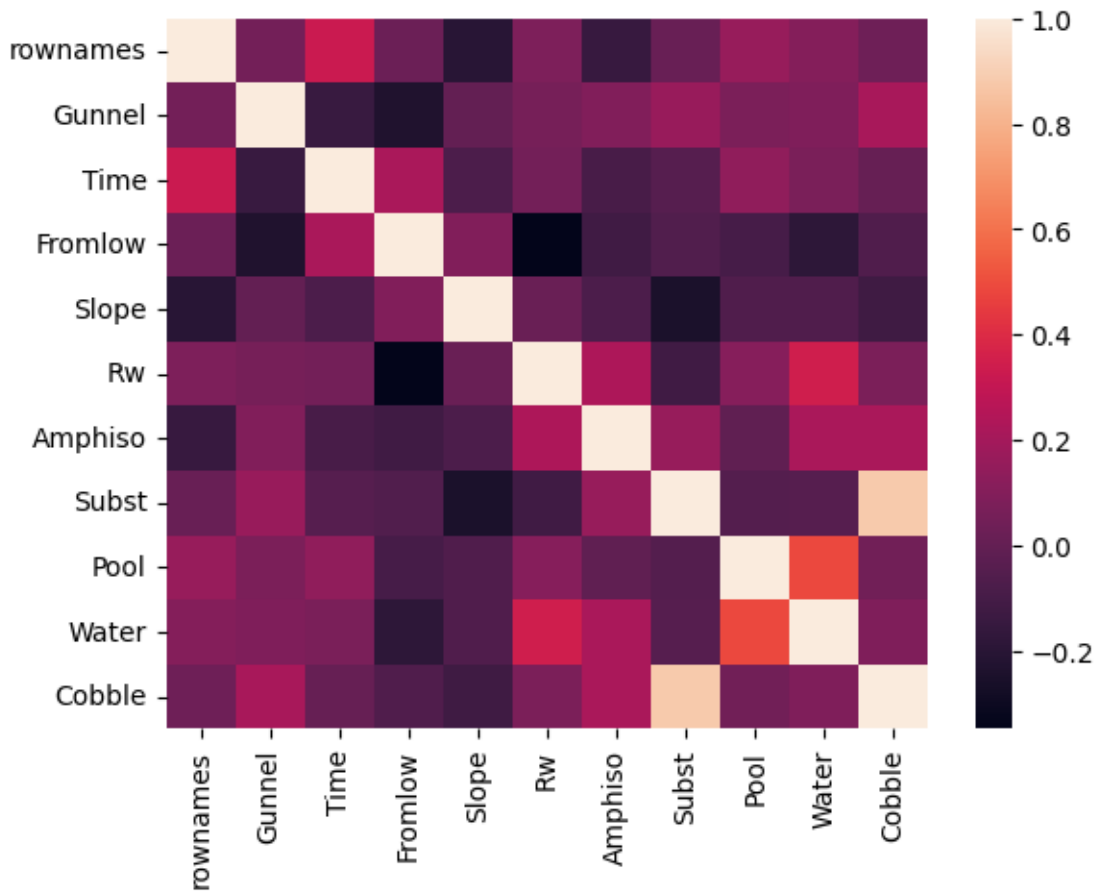
	rownames	Gunnel	Time	Fromlow	Slope	Rw	\
rownames	1.000000	0.049300	0.322613	0.021956	-0.201555	0.080318	
Gunnel	0.049300	1.000000	-0.147345	-0.237442	-0.003742	0.059315	
Time	0.322613	-0.147345	1.000000	0.215593	-0.076922	0.047812	
Fromlow	0.021956	-0.237442	0.215593	1.000000	0.090586	-0.348612	
Slope	-0.201555	-0.003742	-0.076922	0.090586	1.000000	0.019566	
Rw	0.080318	0.059315	0.047812	-0.348612	0.019566	1.000000	
Amphiso	-0.151166	0.092853	-0.094952	-0.118838	-0.078461	0.230164	
Subst	0.014867	0.167311	-0.045537	-0.061721	-0.250251	-0.118785	
Pool	0.157931	0.075823	0.140445	-0.098623	-0.068931	0.107987	
Water	0.101224	0.085071	0.072215	-0.187175	-0.065498	0.344206	
Cobble	0.030921	0.213397	0.008813	-0.064347	-0.124665	0.075399	

	Amphiso	Subst	Pool	Water	Cobble
rownames	-0.151166	0.014867	0.157931	0.101224	0.030921
Gunnel	0.092853	0.167311	0.075823	0.085071	0.213397
Time	-0.094952	-0.045537	0.140445	0.072215	0.008813
Fromlow	-0.118838	-0.061721	-0.098623	-0.187175	-0.064347

Slope	-0.078461	-0.250251	-0.068931	-0.065498	-0.124665
Rw	0.230164	-0.118785	0.107987	0.344206	0.075399
Amphiso	1.000000	0.159754	-0.014739	0.219490	0.220282
Subst	0.159754	1.000000	-0.049367	-0.043498	0.883061
Pool	-0.014739	-0.049367	1.000000	0.488495	0.045803
Water	0.219490	-0.043498	0.488495	1.000000	0.084158
Cobble	0.220282	0.883061	0.045803	0.084158	1.000000

```
[66]: sns.heatmap(correlation,xticklabels=correlation.columns,
                yticklabels=correlation.columns) # Water and Pool do have
                ↪ correlation, but Cobble and Subst have more correlation
```

```
[66]: <Axes: >
```



```
[ ]:
```

Module 5

November 9, 2024

```
[51]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
[52]: gunnels = pd.read_csv("https://raw.githubusercontent.com/salemprakash/EDA/main/
↳Data/Gunnels.csv")
gunnels.head()
```

```
[52]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	\
0	1	0	786	190	10	0.1	0	3	0	0	
1	2	0	794	182	10	0.0	0	3	0	0	
2	3	0	798	178	20	0.0	0	1	0	0	
3	4	0	800	176	70	0.3	0	1	0	1	
4	5	0	804	172	40	0.3	0	1	0	1	

	Cobble
0	0
1	0
2	0
3	0
4	0

```
[53]: X = gunnels.loc[:,["Time", "Slope"]]
X = X.loc[X.Slope > 30,:]
X = X.sample(50, random_state=42)
X.head()
```

```
[53]:
```

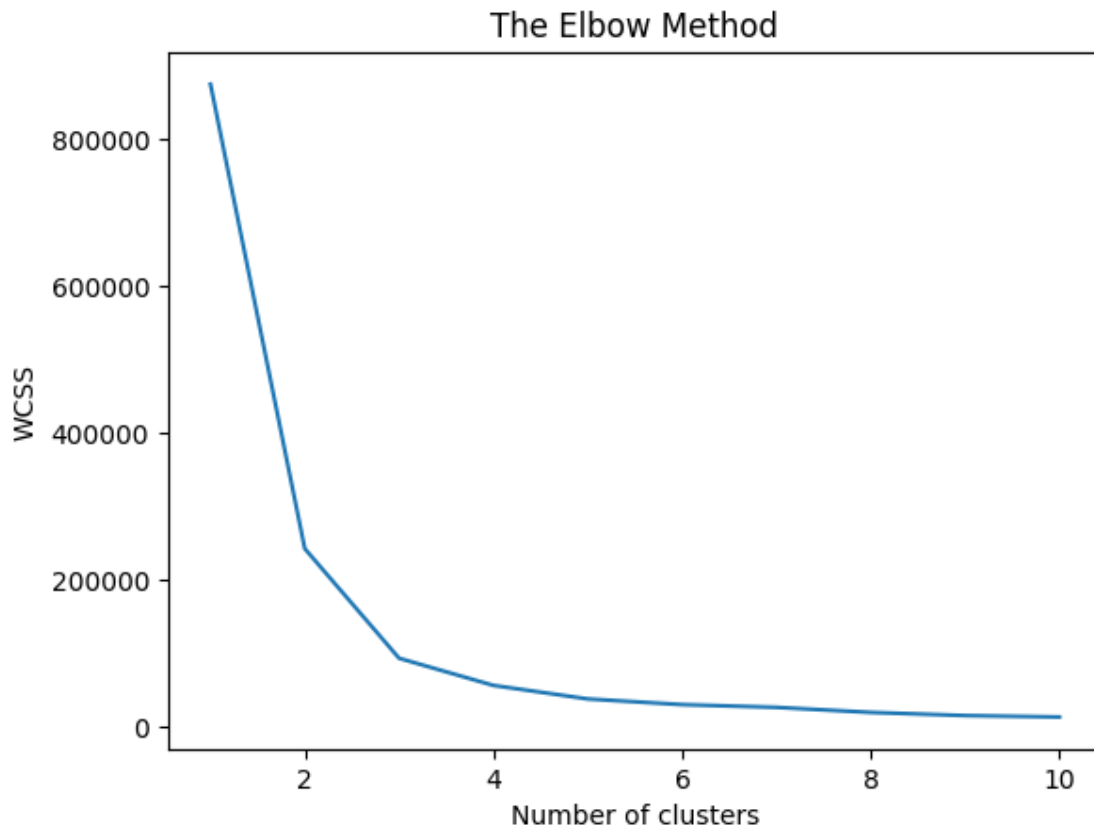
	Time	Slope
412	787	80
215	636	90
631	682	40
904	552	40
3	800	70

```
[54]: from sklearn.cluster import KMeans
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters = i, init = 'k-means++', random_state = 42)
```

```

kmeans.fit(X)
wcss.append(kmeans.inertia_)
plt.plot(range(1, 11), wcss)
plt.title('The Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()

```



```

[55]: kmeans = KMeans(n_clusters = 3, init = 'k-means++', random_state = 42)
      y_kmeans = kmeans.fit_predict(X)

```

```

[56]: y_kmeans

```

```

[56]: array([0, 2, 2, 1, 0, 1, 0, 1, 1, 2, 0, 2, 0, 0, 1, 0, 2, 1, 0, 1, 2, 1,
          2, 2, 2, 1, 1, 1, 2, 2, 1, 1, 0, 0, 0, 2, 0, 2, 0, 2, 1, 2, 2, 1,
          2, 2, 1, 0, 2, 0], dtype=int32)

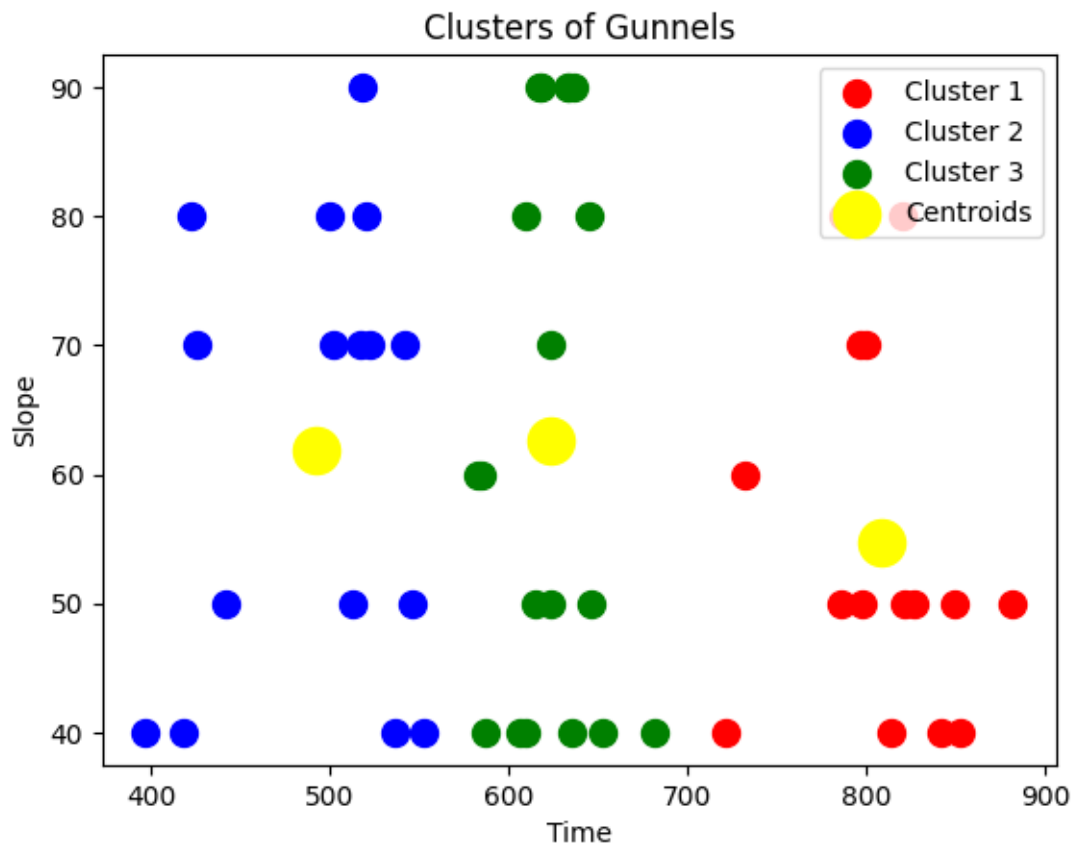
```

```

[57]: X = X.values
      plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label_
      ↪= 'Cluster 1')

```

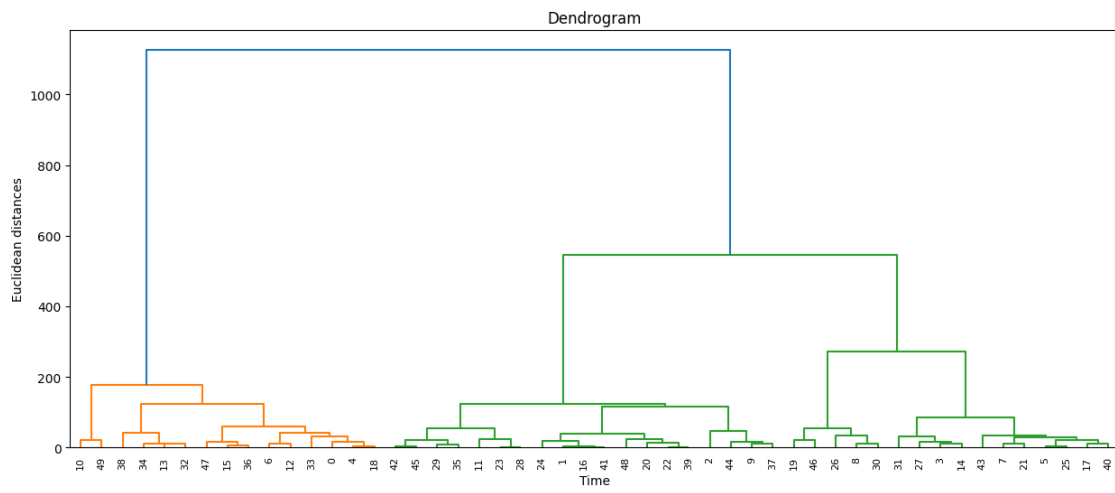
```
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue',
            label = 'Cluster 2')
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green',
            label = 'Cluster 3')
plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s =
            300, c = 'yellow', label = 'Centroids')
plt.title('Clusters of Gunnels')
plt.xlabel('Time')
plt.ylabel('Slope')
plt.legend()
plt.show()
```



```
[58]: import scipy.cluster.hierarchy as sch

plt.figure(figsize=(15,6))
plt.title('Dendrogram')
plt.xlabel('Time')
plt.ylabel('Euclidean distances')
dendrogram = sch.dendrogram(sch.linkage(X, method = 'ward'))
```

```
plt.show()
```



Module 7

November 9, 2024

```
[30]: # Importing the necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[31]: # Load the Gunnels.csv dataset
data = pd.read_csv("https://raw.githubusercontent.com/salemprakash/EDA/main/
↳Data/Gunnels.csv")

# Display the first few rows to confirm loading
data.head()
```

```
[31]:
```

	rownames	Gunnel	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool	Water	\
0	1	0	786	190	10	0.1	0	3	0	0	
1	2	0	794	182	10	0.0	0	3	0	0	
2	3	0	798	178	20	0.0	0	1	0	0	
3	4	0	800	176	70	0.3	0	1	0	1	
4	5	0	804	172	40	0.3	0	1	0	1	

	Cobble
0	0
1	0
2	0
3	0
4	0

```
[32]: from sklearn.model_selection import train_test_split

# Select target and features
target = 'Gunnel'
features = ['Time', 'Fromlow', 'Slope', 'Rw', 'Amphiso', 'Subst', 'Pool']

# Split data into features and target
X = data[features]
y = data[target]

# Train-test split
```

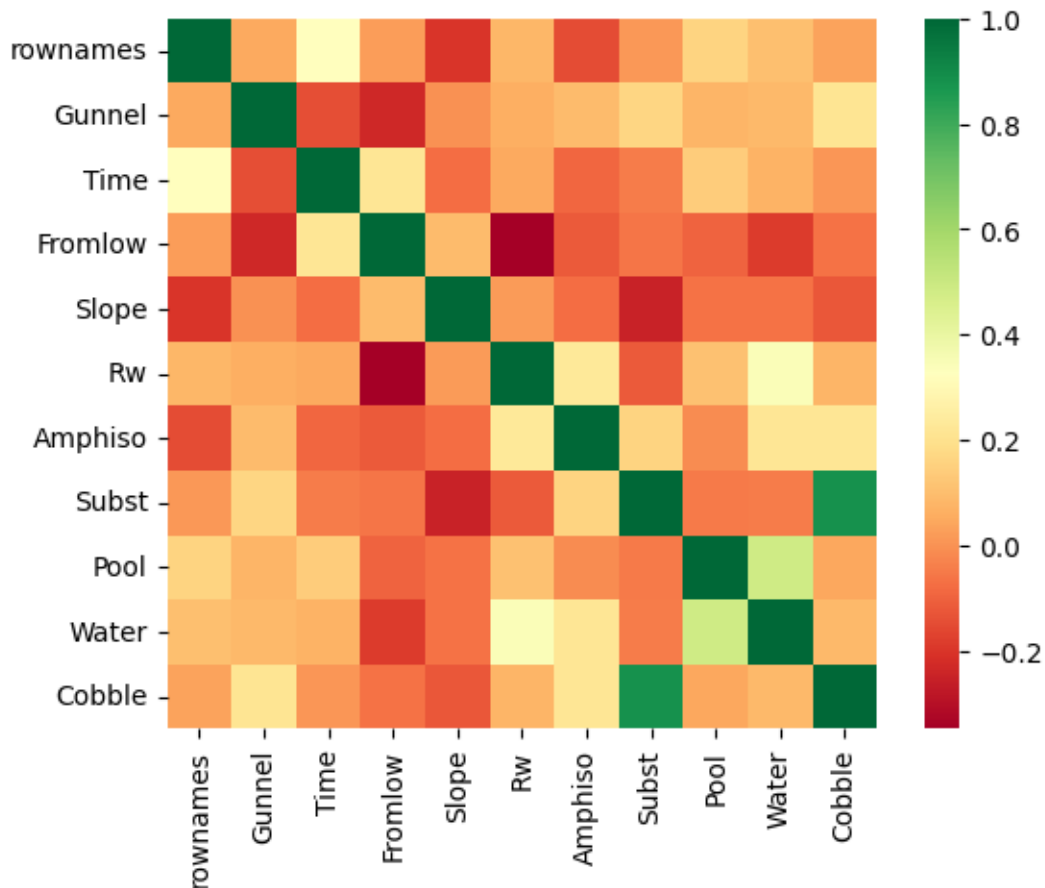
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)
X_train.head()
```

```
[32]:
```

	Time	Fromlow	Slope	Rw	Amphiso	Subst	Pool
1545	835	239	10	0.0	0	8	0
1509	625	29	0	1.0	0	3	1
744	706	226	10	0.0	0	13	0
462	653	56	20	0.9	0	1	0
1356	816	203	0	1.0	0	1	1

```
[33]: #plotting heatmap for overall data set
sns.heatmap(data.corr(), square=True, cmap='RdYlGn')
```

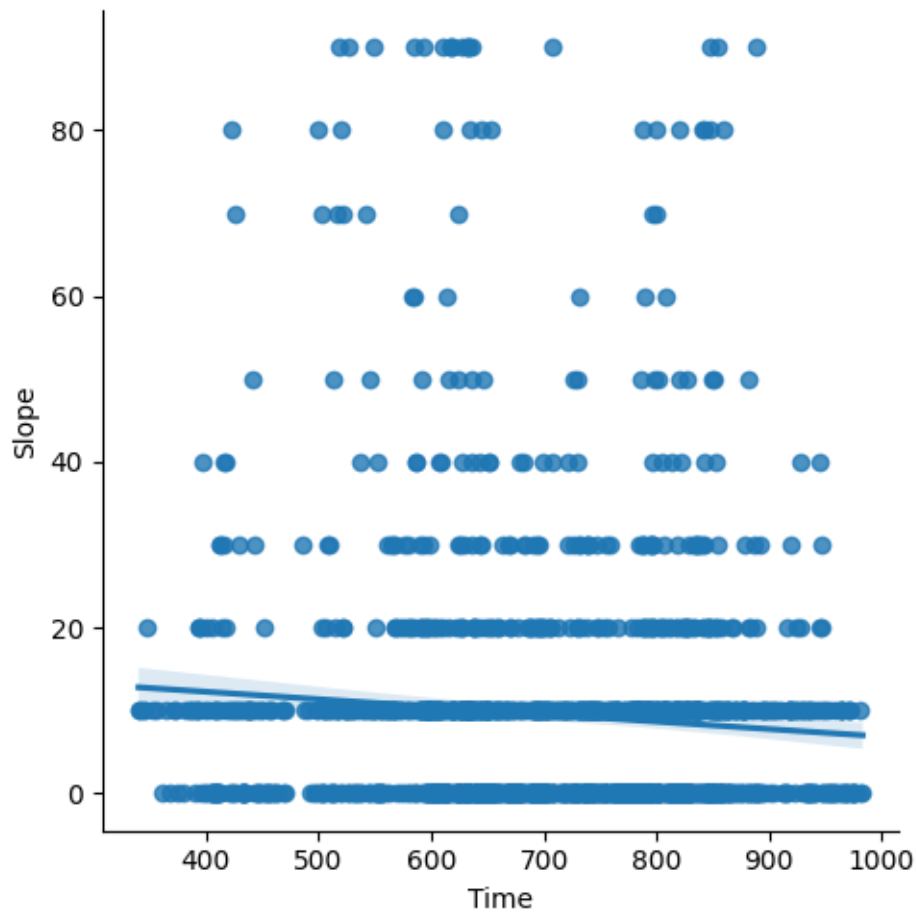
```
[33]: <Axes: >
```



```
[34]: #We can use the lmplo 6. method from seaborn to see the relationship between
↳RM and MEDV. Check out the following snippet:
```

```
sns.lmplot(x = 'Time', y = 'Slope', data = data)
```

[34]: <seaborn.axisgrid.FacetGrid at 0x744a61f94920>



```
[35]: #Now the final step is training the linear regression model. From the extremely
      ↪powerful sklearn library, we import the LinearRegression() function to fit
      #our training dataset to the model. When we run LinearRegression().fit(), the
      ↪function automatically calculates the OLS, which we discussed earlier, and
      #generates an appropriate line function:
      #Training a Linear Regression Model
      from sklearn.linear_model import LinearRegression
      regressor = LinearRegression()
      # Fitting the training data to our model
      regressor.fit(X_train, y_train)
```

[35]: LinearRegression()

```
[36]: #Model evaluation
#R2 can be determined using our test dataset in the LinearRegression.score()
↳method:
#check prediction score/accuracy
regressor.score(X_test, y_test)
```

```
[36]: 0.061134288353554056
```

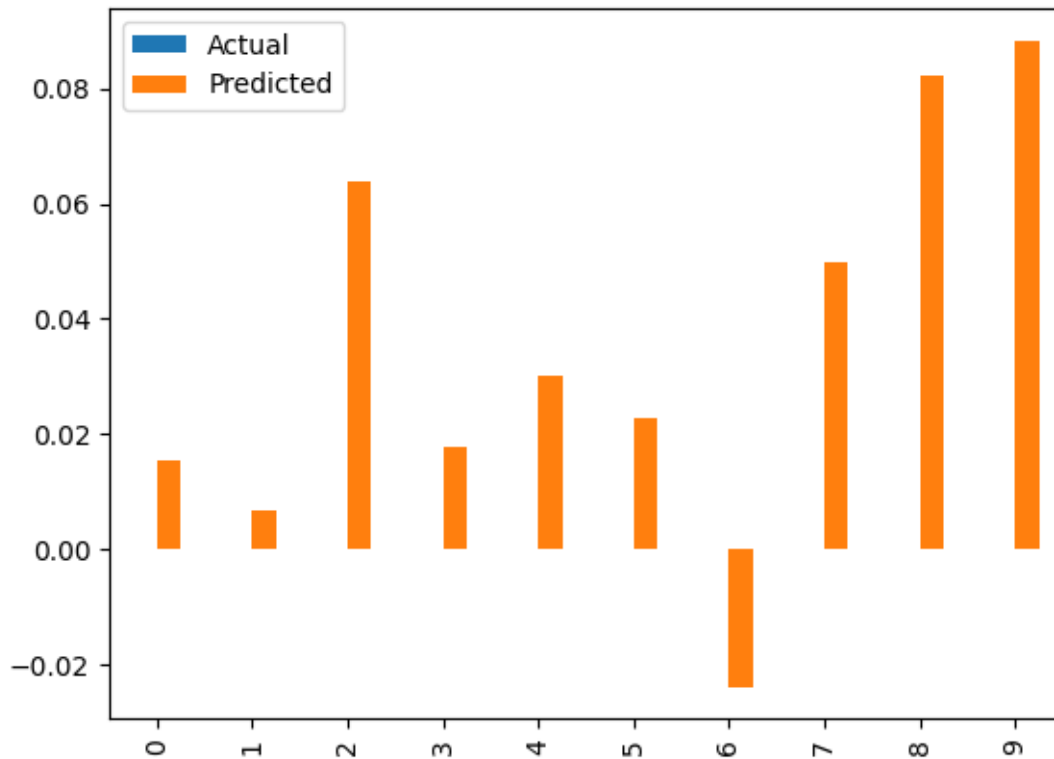
```
[37]: #Before that, let's predict the y values with our model and evaluate it more.
↳And a target variables DataFrame is also built:
# predict the y values
y_pred=regressor.predict(X_test)
# a data frame with actual and predicted values of y
evaluate = pd.DataFrame({'Actual': y_test.values.flatten(),
'Predicted': y_pred.flatten()})
evaluate.head(10)
```

```
[37]:
```

	Actual	Predicted
0	0	0.015465
1	0	0.006812
2	0	0.063908
3	0	0.017676
4	0	0.030019
5	0	0.022660
6	0	-0.023905
7	0	0.049760
8	0	0.082224
9	0	0.088410

```
[38]: #The preceding screenshot shows the difference between the actual values and
↳the predicted values. We can see them if we plot them:
evaluate.head(10).plot(kind = 'bar')
```

```
[38]: <Axes: >
```



```
[39]: #Computing accuracy
# Scoring the model
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
# R2 Score
print(f"R2 score: {r2_score(y_test, y_pred)}")
# Mean Absolute Error (MAE)
print(f"MAE score: {mean_absolute_error(y_test, y_pred)}")
# Mean Squared Error (MSE)
print(f"MSE score: {mean_squared_error(y_test, y_pred)}")
```

R2 score: 0.061134288353554056

MAE score: 0.07240795976383939

MSE score: 0.028508908609265998